Complete Project Files

Tremendous Prepaid Card Extractor

Architecture: Nodriver + Xvfb + Real Chrome

Target Environment: Streamlit Community Cloud

---

1. Main Application: `app.py`

This is the complete, single-file Python script. It dynamically installs Chrome, sets up the Xvfb virtual display, bypasses Cloudflare Turnstile using CDP, and renders the Streamlit UI.

```
import asyncio
import os
import re
import tempfile
import traceback
import subprocess
import openpyxl
import streamlit as st
import pandas as pd
from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill, Alignment
from openpyxl.utils import get_column_letter
import nodriver as uc
import nest_asyncio

# --- FIX: PREVENT PYTHON 3.14 EVENT LOOP CRASH ---
try:
    loop = asyncio.get_running_loop()
    nest_asyncio.apply()
except RuntimeError:
    pass

INTER_REQUEST_DELAY_SECONDS = 2

# --- ENVIRONMENT SETUP (Chrome + Xvfb) ---
@st.cache_resource(show_spinner=False)
def setup_environment():
    st.info("Initializing Virtual Display and Chrome...")

    # 1. Start Xvfb (Virtual Display)
    if not os.path.exists("/tmp/.X99-lock"):
        subprocess.Popen(["Xvfb", ":99", "-screen", "0", "1920x1080x24"], stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
    os.environ['DISPLAY'] = ':99'

    # 2. Download and Extract Google Chrome (No root required)
    chrome_dir = "/tmp/chrome"
    chrome_bin = os.path.join(chrome_dir, "opt/google/chrome/google-chrome")
```

```
    if not os.path.exists(chrome_bin):
        os.system("wget -q -O /tmp/chrome.deb https://dl.google.com/linux/direct/google-chrome-
stable_current_amd64.deb")
        os.system(f"dpkg -x /tmp/chrome.deb {chrome_dir}")
        os.system("rm /tmp/chrome.deb")
    return chrome_bin

CHROME_PATH = setup_environment()

# --- SESSION STATE INIT ---
if "targets" not in st.session_state: st.session_state.targets = None
if "results" not in st.session_state: st.session_state.results = None
if "uploaded_file" not in st.session_state: st.session_state.uploaded_file = None
if "processing_complete" not in st.session_state: st.session_state.processing_complete = False
if "file_name" not in st.session_state: st.session_state.file_name = None

# --- HELPERS ---
def normalize_cell_value(value):
    if value is None: return ""
    if isinstance(value, int): return str(value)
    if isinstance(value, float): return str(int(value)) if value.is_integer() else str(value)
    return str(value).strip()

def get_hyperlink_or_url(cell):
    try:
        if cell.hyperlink and cell.hyperlink.target: return str(cell.hyperlink.target).strip()
    except: pass
    value = cell.value
    if value is None: return None
    value_str = str(value).strip()
    if value_str.lower().startswith(("http://", "https://")): return value_str
    return None

def extract_balance(text):
    if not text: return None
    t = text.replace("\u00a0", " ")
    m = re.search(r"Available\s+Balance[\s\S]{0,50}?\$\s*([\d,]+\.\d{2})", t, re.I)
    if m: return f"{float(m.group(1).replace(',', '')):.2f}"
    return None

# --- NODRIVER LOGIC ---
async def process_one_url(driver, url):
    if not url: return None, "No URL"
    try:
        tab = await driver.get(url)
    except Exception:
        raise Exception("DriverDead")
    try:
        await tab.sleep(3)
        try:
            page_title = await tab.evaluate("document.title")
        except: page_title = ""
        if "Just a moment" in page_title or "Performing security verification" in page_title:
            try:
                await tab.verify_cf()
                await tab.sleep(5)
            except: pass
        balance = None
        for _ in range(15):
            try:
                page_text = await tab.evaluate("document.body ? document.body.innerText : ''")
```

```
                balance = extract_balance(page_text)
                if balance: break
            except: pass
            await tab.sleep(1)
        return (balance, "Success") if balance else (None, "Balance Not Found")
    except Exception:
        raise Exception("DriverDead")

async def start_browser(executable_path):
    return await uc.start(
        browser_executable_path=executable_path,
        headless=False,
        browser_args=["--window-size=1920,1080", "--no-sandbox", "--disable-dev-shm-usage"]
    )

async def run_processor(targets, progress_callback, chrome_path):
    results = []
    total = len(targets)
    driver = await start_browser(chrome_path)
    try:
        for idx, target in enumerate(targets, 1):
            progress_callback(idx, total, target["reward_id"])
            url = target["url"]
            if not url:
                results.append({**target, "balance": "N/A", "status": "No URL"})
                continue
            try:
                balance, status = await process_one_url(driver, url)
            except Exception:
                try: driver.stop()
                except: pass
                await asyncio.sleep(2)
                driver = await start_browser(chrome_path)
                try: balance, status = await process_one_url(driver, url)
                except: balance, status = None, "Crash Retry Failed"
            results.append({
                **target,
                "balance": balance if balance is not None else "N/A",
                "status": status
            })
            if idx < total: await asyncio.sleep(INTER_REQUEST_DELAY_SECONDS)
    finally:
        driver.stop()
    return results

# --- EXCEL EXPORT ---
def export_to_excel(results):
    wb = Workbook()
    ws = wb.active
    ws.title = "Balances"
    headers = ["Reward ID", "Card Number", "Extracted Balance", "Overall Status"]
    ws.append(headers)

    header_fill = PatternFill(start_color="1F4E78", end_color="1F4E78", fill_type="solid")
    header_font = Font(bold=True, color="FFFFFF")
    link_font = Font(color="0563C1", underline="single")

    for col in range(1, len(headers)+1):
        cell = ws.cell(row=1, column=col)
        cell.fill = header_fill
        cell.font = header_font
```

```
            cell.alignment = Alignment(horizontal="center", vertical="center")

        ws.freeze_panes = "A2"

        for row_idx, item in enumerate(results, start=2):
            cell_a = ws.cell(row=row_idx, column=1)
            cell_a.value = item["reward_id"]
            if item.get("url"):
                cell_a.hyperlink = item["url"]
                cell_a.font = link_font

            cell_b = ws.cell(row=row_idx, column=2)
            cell_b.value = item["card_number"]
            if item.get("card_url"):
                cell_b.hyperlink = item["card_url"]
                cell_b.font = link_font

            ws.cell(row=row_idx, column=3).value = item["balance"]
            ws.cell(row=row_idx, column=4).value = item["status"]

        with tempfile.NamedTemporaryFile(delete=False, suffix=".xlsx") as tmp:
            wb.save(tmp.name)
            return tmp.name

    # --- STREAMLIT UI ---
    st.set_page_config(page_title="Tremendous Extractor", layout="wide")
    st.title("🗃️ Tremendous Balance Extractor")
    st.markdown("Upload your Excel file (Columns: **A = Reward ID**, **B = Card Number**) and click
    **Process**.")

    col_clear, _ = st.columns([1, 5])
    with col_clear:
        if st.button("🗑️ Clear All Data", use_container_width=True):
            for key in ["targets", "results", "uploaded_file", "processing_complete", "file_name"]:
                if key in st.session_state: del st.session_state[key]
            st.rerun()

    uploaded_file = st.file_uploader("Choose an Excel file", type=["xlsx"], key="file_uploader")

    if uploaded_file is not None:
        file_bytes = uploaded_file.read()
        if st.session_state.uploaded_file != file_bytes:
            st.session_state.uploaded_file = file_bytes
            st.session_state.file_name = uploaded_file.name
            st.session_state.targets = None
            st.session_state.results = None
            st.session_state.processing_complete = False

        if st.session_state.targets is None:
            with tempfile.NamedTemporaryFile(delete=False, suffix=".xlsx") as tmp_input:
                tmp_input.write(st.session_state.uploaded_file)
                input_path = tmp_input.name

            wb_input = openpyxl.load_workbook(input_path, data_only=False)
            sheet = wb_input["Main List"] if "Main List" in wb_input.sheetnames else wb_input.active
            targets = []
            for row_num, row in enumerate(sheet.iter_rows(min_row=2, max_col=2), start=2):
                cell_a, cell_b = row[0], row[1]
                if cell_a.value is None and cell_b.value is None: continue
                reward_id = normalize_cell_value(cell_a.value) or "N/A"
                card_number = normalize_cell_value(cell_b.value) or "N/A"
```

```
            reward_url = get_hyperlink_or_url(cell_a)
            card_url = get_hyperlink_or_url(cell_b)
            if not reward_url and card_url: reward_url = card_url
            targets.append({
                "row": row_num, "reward_id": reward_id, "card_number": card_number,
                "url": reward_url, "card_url": card_url,
            })
        wb_input.close()
        os.unlink(input_path)
        st.session_state.targets = targets
        st.session_state.processing_complete = False

    st.success(f"✅ Loaded {len(st.session_state.targets)} records from {st.session_state.file_name}")

    if not st.session_state.processing_complete:
        if st.button("🚀 Process All URLs", type="primary", use_container_width=True):
            targets = st.session_state.targets
            progress_bar = st.progress(0)
            status_text = st.empty()

            def progress_callback(current, total, reward_id):
                progress_bar.progress(current / total)
                status_text.text(f"Processing {current}/{total}: {reward_id[:30]}...")

            results = [] # PREVENTS NameError CRASH
            loop = asyncio.new_event_loop()
            asyncio.set_event_loop(loop)
            try:
                results = loop.run_until_complete(run_processor(targets, progress_callback, CHROME_PATH))
            except Exception as e:
                st.error(f"❌ An error occurred: {e}\n\n{traceback.format_exc()}")
            finally:
                loop.close()

            if results:
                st.session_state.results = results
                st.session_state.processing_complete = True
            else:
                st.error("The browser failed to launch or returned no results.")

            progress_bar.empty()
            status_text.empty()
            st.rerun()

    if st.session_state.results is not None and st.session_state.processing_complete:
        results = st.session_state.results
        st.subheader("📊 Results Summary")
        success = sum(1 for r in results if r["status"] == "Success")
        errors = sum(1 for r in results if "Error" in r["status"] or "Crash" in r["status"])

        col1, col2 = st.columns(2)
        col1.metric("✅ Success", success)
        col2.metric("❌ Errors", errors)

        df = pd.DataFrame(results)
        st.dataframe(df[["reward_id", "card_number", "balance", "status"]])

        excel_path = export_to_excel(results)
        with open(excel_path, "rb") as f:
            st.download_button(
                label="📥 Download Report (Excel)",
```

```
                data=f,
                file_name="Tremendous_Balances_Report.xlsx",
                mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
                use_container_width=True
            )
        os.unlink(excel_path)

        if st.button("🔄 Process Again", use_container_width=True):
            st.session_state.results = None
            st.session_state.processing_complete = False
            st.rerun()
```

2. Python Dependencies: `requirements.txt`

These are the required Python packages. Note that we use `nodriver` instead of standard automation tools like Playwright to successfully bypass Cloudflare.

```
streamlit
openpyxl
pandas
nodriver
nest-asyncio
```

3. Linux System Dependencies: `packages.txt`

Streamlit Cloud runs on Debian Linux. Google Chrome requires these X11 and graphical libraries to run properly inside the Xvfb virtual display.

```
xvfb
wget
dpkg
fonts-liberation
libnss3
libx11-xcb1
libxcb1
libxcomposite1
libxcursor1
libxdamage1
libxi6
libxtst6
libxrandr2
libasound2
libatk1.0-0
libatk-bridge2.0-0
libgtk-3-0
libgbm1
libpango-1.0-0
libcairo2
libgdk-pixbuf-2.0-0
libxkbcommon0
```

```
libxss1
libxfixes3
libxinerama1
libxrender1
libxshmfence1
libxxf86vm1
libgl1
libgl1-mesa-dri
libdrm2
libgles2
libegl1
libegl-mesa0
```

4. Streamlit Configuration: `.streamlit/config.toml`

Create a folder named `.streamlit` in your GitHub repository and place this file inside it. It increases the maximum upload size to 200MB.

```
[server]
maxUploadSize = 200
```

5. Deployment Guide: `README.md`

This documentation explains the project structure and deployment steps.

```
# Tremendous Balance Extractor

This Streamlit application automatically extracts financial balances from Tremendous Prepaid Cards. It
completely bypasses Cloudflare's Turnstile security checks using Nodriver and a virtual display (Xvfb).

## Features
- **Cloudflare Bypass:** Uses Nodriver (CDP) and Headful Chrome to appear as a real human user.
- **Zero Root Required:** Automatically downloads and extracts Google Chrome to a local folder
(`/tmp/chrome`) on startup.
- **Crash Recovery:** If the browser crashes during a batch process, it restarts automatically and
resumes from the failed URL.
- **Excel Formatting:** Outputs a beautifully styled `.xlsx` report with clickable hyperlinks.

## Required Files
To deploy this app, ensure your GitHub repository has the following files:
1. `app.py` (The main Streamlit code)
2. `requirements.txt` (Python packages)
3. `packages.txt` (Linux system dependencies for Chrome/Xvfb)
4. `.streamlit/config.toml` (Upload limit settings)
5. `README.md` (Documentation)

## Deployment (Streamlit Cloud)
1. Push the files to a public GitHub repository.
2. Log in to [share.streamlit.io](https://share.streamlit.io).
3. Click "New app", select your repo and the `main` branch.
```

```
4. Set the Main file path to `app.py`.
5. Click **Deploy**.

## How to use
1. Open the deployed Streamlit URL.
2. Upload an Excel file with Reward IDs in Column A and Card Numbers in Column B.
3. Click "Process All URLs".
4. Wait for the progress bar to complete.
5. Download the generated Excel report.

## Troubleshooting
- **Error: Chrome not found:** Streamlit Cloud occasionally blocks the dynamic download of Chrome. Ensure `wget` and `dpkg` are listed in `packages.txt`.
- **Error: Results not defined:** This was fixed by initializing `results = []` before the async loop.
- **Blocked by Cloudflare:** Ensure the `nodriver` package is up to date. Run `pip install --upgrade nodriver` and push the changes.
```